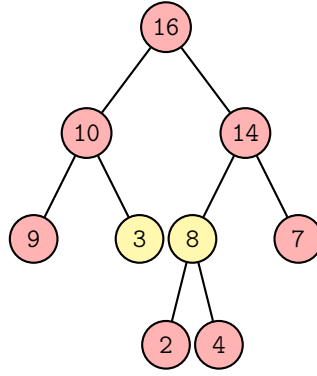




Esercizio B1

Enumerare tutti i percorsi è un problema di natura potenzialmente superpolinomiale; va quindi risolto tramite backtrack.

L'idea è simile a quanto visto in altri compiti: utilizziamo la ricorsione per visitare, in profondità, la prossima mossa sulla matrice. Utilizziamo una matrice *visited* per segnare le caselle già visitate; *visited*[*i*][*j*] sarà posta a **true** quando una cella viene visitata, per poi ritornare a **false** al completamento della visita.

Se arriviamo alla casella finale (*n*, *n*), ritorniamo il valore 1, per contare un cammino semplice che termina in (*n*, *n*). Altrimenti, contiamo quanti cammini possono essere ottenuti andando nelle quattro direzioni permesse. Se (*i*, *j*) non è una cella valida oppure è già stata visitata, ritorniamo 0.

La complessità dell'algoritmo proposto è limitata superiormente da $O(4^{n^2})$; è possibile notare che a meno di valori tutti pari a 1, gran parte dell'albero della visita viene potato dal fatto che molti salti non sono possibili.

```
int countPaths(int[][] A, int n)
```

```
    boolean[][] visited = new boolean[1...n][1...n] = { false }           % Initialized to false
    return countRec(A, visited, n, 1, 1)
```

```
int countRec(int[][] A, boolean[][] visited, int n, int i, int j)
```

```
    if i==n and j==n then
        return 1
    else if 1 ≤ i ≤ n and 1 ≤ j ≤ n and not visited[i][j] then
        visited[i][j] = true
        int ret = countRec(A, visited, n, i + A[i][j], j) +
                  countRec(A, visited, n, i, j + A[i][j]) +
                  countRec(A, visited, n, i - A[i][j], j) +
                  countRec(A, visited, n, i, j - A[i][j])
        visited[i][j] = false
        return ret
    else
        return 0
```

Ho visto alcuni tentativi di risolvere il problema tramite programmazione dinamica. Senza entrare nei dettagli, molte di queste soluzioni (i) si "muovevano" avanti e indietro nello spazio della matrice e (ii) non tenevano conto delle celle già visitate lungo un cammino. In altre parole, non avevano la proprietà di sottostruttura ottima necessaria per poter applicare la programmazione dinamica.

Esercizio B2

Il problema può essere risolto trasformandolo in un'istanza di Somma Massimale, il primo algoritmo visto a lezione. Si crea un vettore di supporto che contiene 1 per ogni valore 0, -1 per ogni valore 1. In questo modo, la somma massimale in un vettore corrisponde allo zero-sbilanciamento dello stesso.

Il costo dell'algoritmo è quindi $\Theta(n)$.

```
int zeroUnbalance(int[] A, int n)
```

```
    int[] B = new int[1...n]
    for i = 1 to n do
        B[i] = iff(A[i] == 0, 1, -1)
    return maxsum(B, n)
```

Vorrei comunque far notare che qualunque degli approcci visti per somma massimale possono essere utilizzati per creare una soluzione ad-hoc per il problema. Ad esempio, la soluzione seguente ha costo $\Theta(n^3)$ e dovrebbe essere proposta da chiunque abbia completato con successo un corso di Programmazione 1.

```
int zeroUnbalance(int[] A, int n)
```

```
    int maxSoFar = 0
    for i = 1 to n do
        for j = i to n do
            int n1 = sum(A, i, j)                % Number of ones in A[i...j], computational cost O(n)
            int n0 = (j - i + 1) - n1            % Number of zeros in A[i...j]
            int unbalance = n0 - n1
            maxSoFar = max(maxSoFar, unbalance)
    return maxSoFar
```

Meglio scrivere una soluzione di questo tipo (che non era esplicitamente vietata) che non scrivere nulla.

Esercizio B3

Anche in questo caso, è possibile scrivere una soluzione molto semplice in $\Theta(n^4)$. Si può fare (leggermente) meglio di così.

Si costruiscano due matrici di supporto R , C di dimensione $n \times n$ tali per cui

- $R[i][j]$ contiene la lunghezza della più lunga sequenza orizzontale di 1 che inizia a sinistra di (i, j) e termina in (i, j) ;
- $C[i][j]$ contiene la lunghezza della più lunga sequenza orizzontale di 1 che inizia sopra di (i, j) e termina in (i, j) .

Questi due vettori vengono facilmente calcolati in tempo $O(n^2)$.

A questo punto, possono essere utilizzati per verificare se esiste un quadrato di dimensione $k \in [1, \dots, \min([R[i][j], C[i][j]])]$, utilizzando le matrici R , C per verificare in tempo $O(1)$ se esistono tutti i lati.

Sono necessari tre cicli annidati e la complessità è quindi $O(n^3)$. Tuttavia, è possibile migliorare leggermente l'algoritmo notando che è inutile verificare quadrati di dimensione $maxSoFar$ o inferiori, dove $maxSoFar$ è il quadrato più grande trovato finora.

```

int largestSquare(int[][] A, int n)


---


  int[][] R = new int[1...n][1...n]
  int[][] C = new int[1...n][1...n]
  for i = 1 to n do
     $\left[ \begin{array}{l} R[i][1] = A[i][1] \\ C[1][i] = A[1][i] \end{array} \right.$ 
  for i = 2 to n do
    for j = 2 to n do
       $\left[ \begin{array}{l} R[i][j] = \text{iif}(A[i][j] == 0, 0, R[i][j-1] + 1) \\ C[i][j] = \text{iif}(A[i][j] == 0, 0, C[i-1][j] + 1) \end{array} \right.$ 
  int maxSoFar = 0
  for i = 1 to n do
    for j = 1 to n do
      int m = min(R[i][j], C[i][j])
      for k = maxSoFar + 1 to m do
         $\left[ \begin{array}{l} \text{if } R[i-k+1][j] \geq k \text{ and } C[i][j-k+1] \geq k \text{ then} \\ \quad \left[ \begin{array}{l} \text{maxSoFar} = \max(\text{maxSoFar}, k) \end{array} \right. \end{array} \right.$ 
  return maxSoFar

```
